Deep Learning

Sungyoon Lee

Department of Computer Science
Hanyang University

sungyoonlee@hanyang.ac.kr

Understanding Language

- NNs for Language
- Transformer
- LLM

Lecture 15

Understanding Language

Understanding Language
NNs for Language
Transformer
LLM

Natural Language Processing (NLP)

- text classification
  - email spam classification
  - sentiment analysis
- machine translation
- speech recognition
- image captioning
- speech tagging
- document summarization
- question answering
- ...

Various Issues in Text Data

- documents have a variable length (unstructured)
  $\rightarrow$ TF-IDF $\in \mathbb{R}^{V \times N}$ (V: # vocab., N: # documents)
  - words are categorical variables with many possible values ($V \gg 1$)
  - one-hot encoding[1] has no natural notion of similarity
    $\rightarrow$ word embedding $\mathbb{R}^V \rightarrow \mathbb{R}^K$ ($K < V$)
- may encounter words at test time that have not been seen during training (so-called out-of-vocabulary or OOV words)
  $\rightarrow$ replace all novel words with the special symbol UNK, which stands for "unknown".[2]

[1] categorical variable to one-hot encoding, e.g., one-hot(red)$= [1, 0, 0]$, one-hot(green)$= [0, 1, 0]$, one-hot(blue)$= [0, 0, 1]$

[2] We could work at the character level. ... subword units, wordpieces, byte-pair encoding

bag of words, term frequency (TF) matrix, document frequency (DF), inverse DF, TF-IDF

For the vocabulary $\{v_i\}_{i=1}^{V}$ and the documents $\{x_n\}_{n=1}^{N}$, we have

$$\mathrm{BOW}_i(x_*)^{3} := \sum_{t=1}^{|x_*|} \mathbf{1}(x_{*t} = v_i) \qquad \text{(how many } v_i\text{'s in } x_*\text{)}$$

$$\mathrm{TF}_{in}{}^{4} := \sum_{t=1}^{|x_n|} \mathbf{1}(x_{nt} = v_i) \qquad \text{(how many } v_i\text{'s in } x_n\text{)}$$

$$\mathrm{DF}_i{}^{5} := \sum_{n=1}^{N} \mathbf{1}(v_i \in x_n) \qquad \text{(how many doc.s have } v_i\text{)}$$

$$\mathrm{IDF}_i := \log\left(\frac{N}{1 + \mathrm{DF}_i}\right)^{6}$$

$$\mathrm{TFIDF}_{in} := \log(\mathrm{TF}_{in} + 1) \times \mathrm{IDF}_i$$

---

[3] BOW$\in \mathbb{R}^V$ (as a structured data); vector space model

[4] $\mathrm{TF} = [\mathrm{BOW}(x_1)\ \mathrm{BOW}(x_2)\ \cdots\ \mathrm{BOW}(x_N)] \in \mathbb{R}^{V \times N}$

[5] DF = the sum of the values in each row of TF

[6] to reduce the impact of words that occurs many times

The TF-IDF transformation does not overcome the fundamental issue that semantically similar words, such as "man" and "woman", and semantically dissimilar words, such as "man" and "banana", have the same distance (Hamming distance of 1).

Word Embeddings

For one-hot vector $x_{nt} \in \{0,1\}^V$[7], we define

$$e_{nt} := Ex_{nt} \in \mathbb{R}^K \qquad \text{(word embeddings)}$$

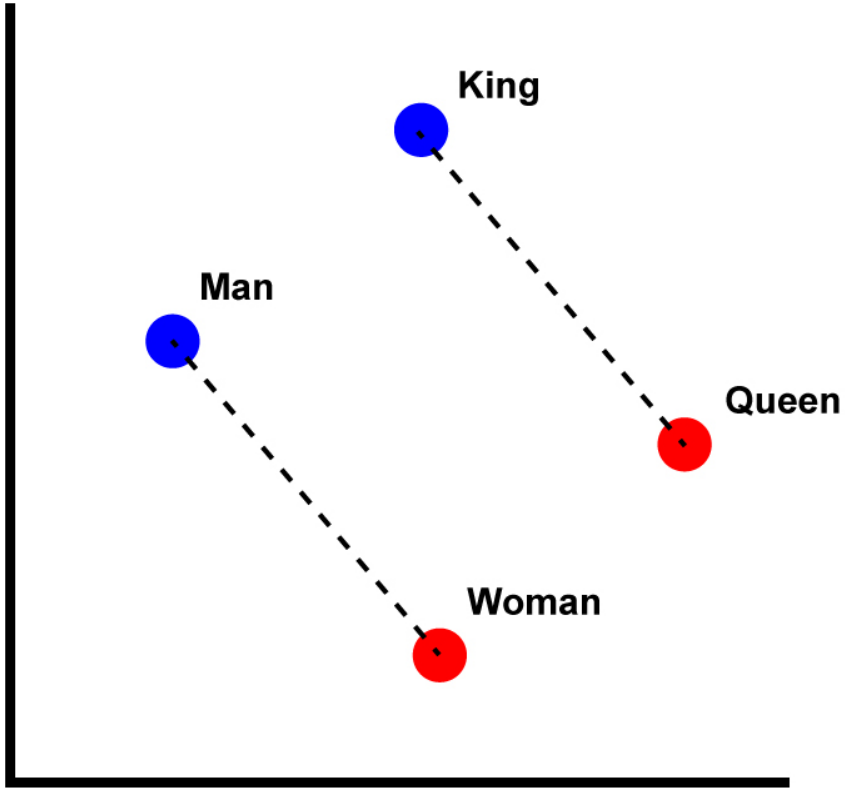
$$\bar{e}_n := \sum_{t=1}^{|x_n|}{}^{8} e_{nt} = E \cdot BOW(x_n) \qquad \text{(bag of word embeddings)}$$

where $E = [e(v_1)\ e(v_2)\ \cdots\ e(v_V)] \in \mathbb{R}^{K \times V}$ and $e(v_i) \in \mathbb{R}^K$

- $K < V$
- e.g., $p(y = c \mid x_n; \theta) = \text{softmax}_c(W\bar{e}_n)$ (multi-class logistic regression)

[7] a standard basis vector

[8] $\sim$ global average pooling



algebraic operations on the vector representations of words (word2vec [MCC+13])

Word Embeddings

semantically similar = occur in similar contexts (distributional hypothesis) = close in vector space

- Word2vec (CBOW, skipgram)
- ...
- contextual word embeddings (ELMo, BERT, GPT, T5, ...): The contextual embedding of a word is a function of all the words in its context (usually a sentence). This can give much improved results, and is currently the standard approach to representing natural language data, as a preprocessing step before doing transfer learning.

Short Introduction to Probabilistic Graphical Models

Probabilistic Graphical Model (PGM)

A PGM is a joint distribution using a graph (DAG) to encode conditionally independent (CI) assumption:

$$Y_i \perp Y_{pred(i) \setminus pa(i)} \mid Y_{pa(i)},$$

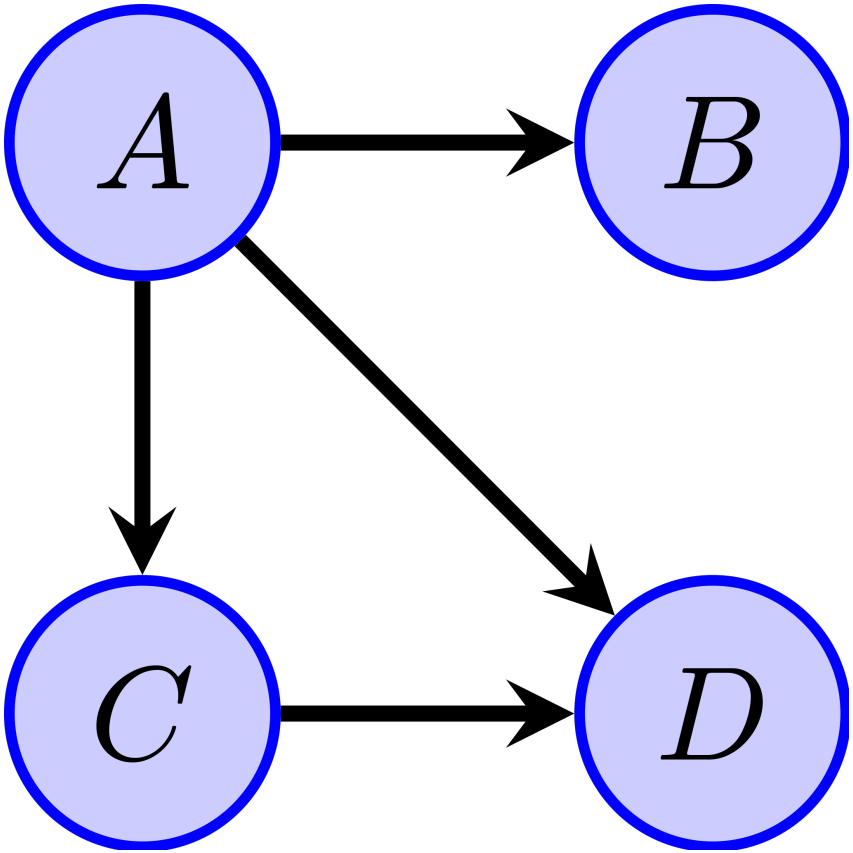
where $pred(i)$ and $pa(i)$ are the predecessors and parents of node i, respectively.

- graph: node (random variable), directed edge (dependency), lack of edge (CI)

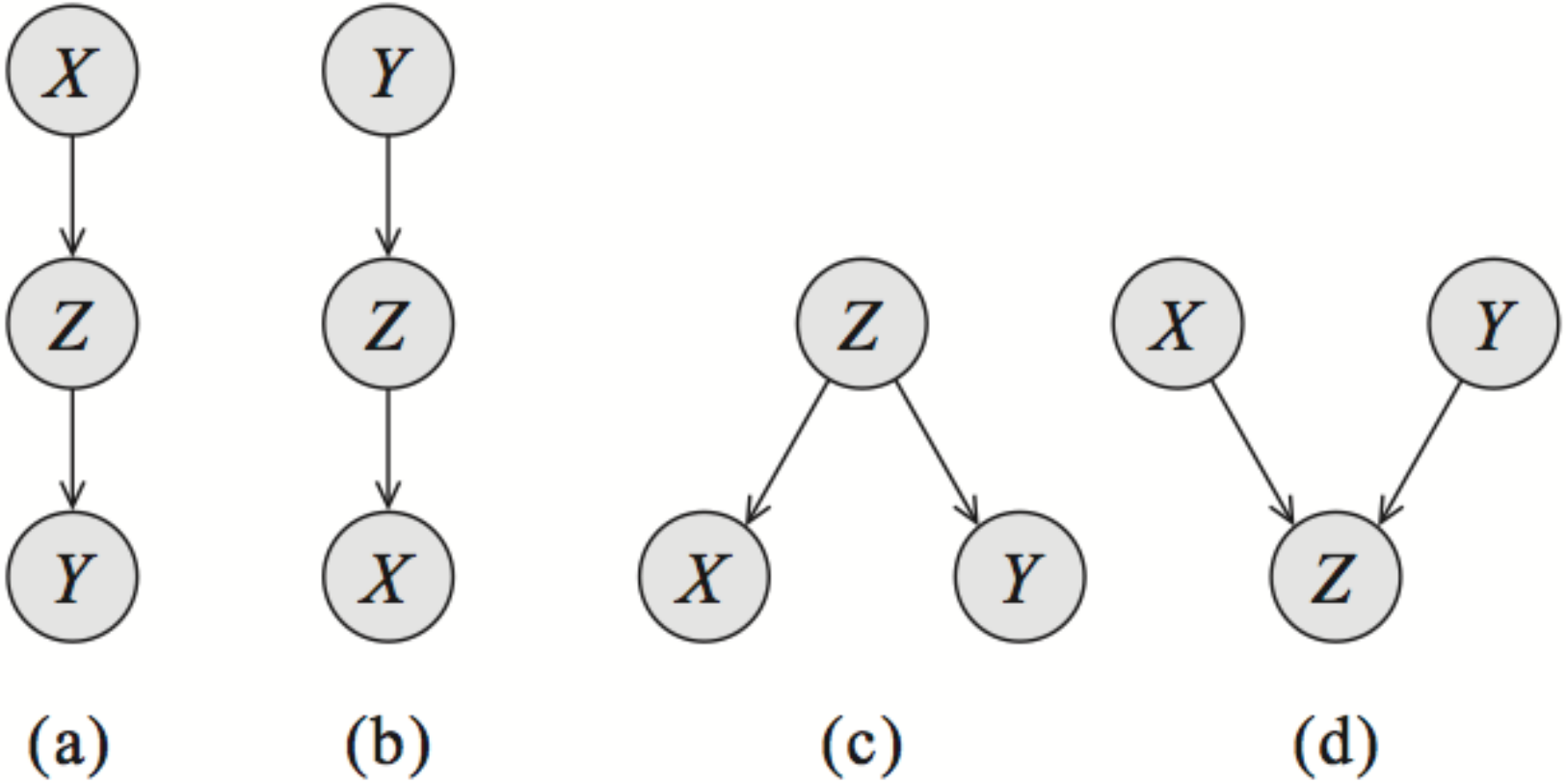
Consequently, we can represent the joint distribution as follows:

$$p(Y_{1:V}) = \prod_{i=1}^{V} p(Y_i \mid Y_{pa(i)})$$

Q. Compute the $p(A, B, C, D)$ for the following graph:

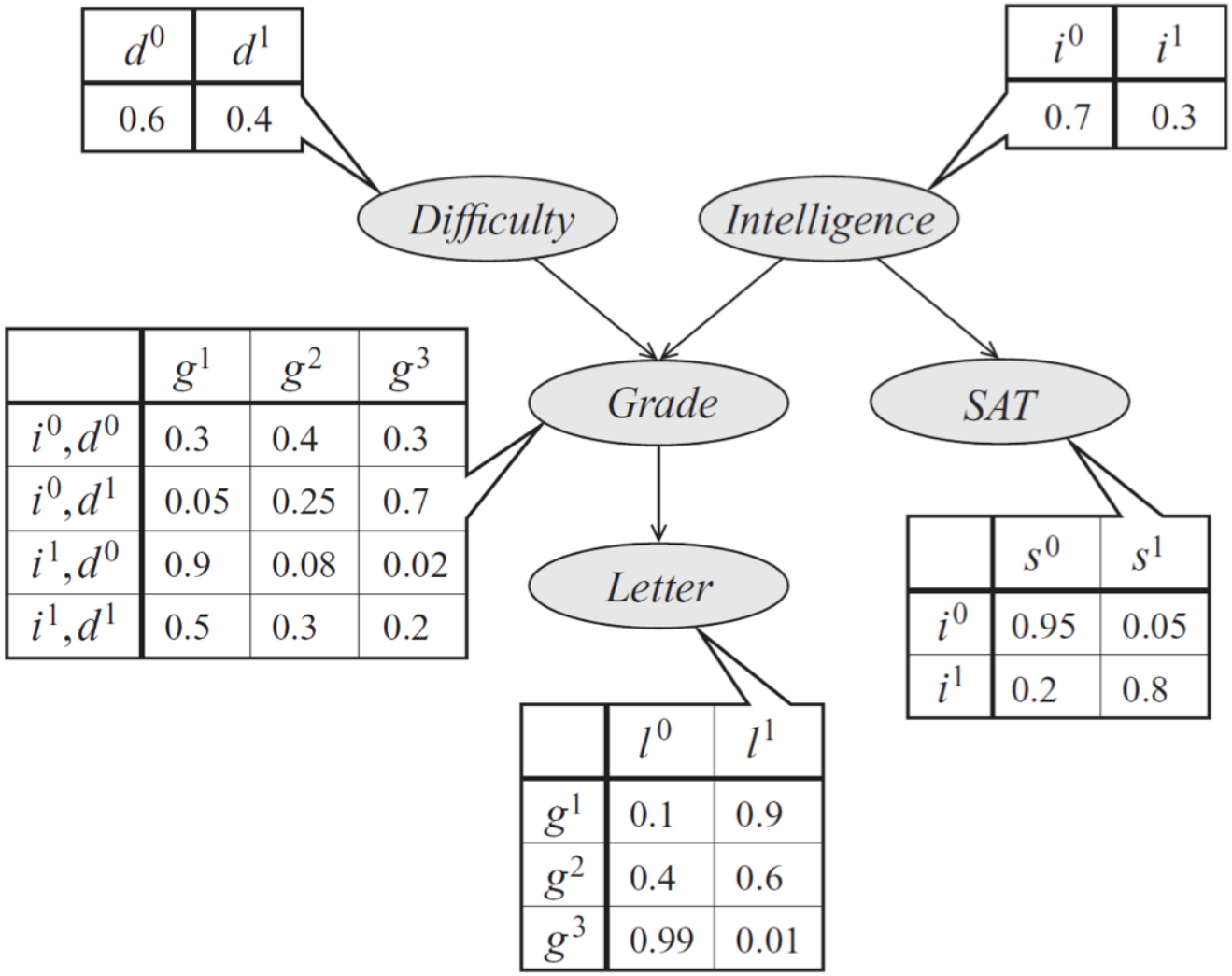


Q. Compute the $p(X, Y, Z)$ for the following graph:



cf. (a,b) $X \perp Y \mid Z$

Q. Compute the $p(D, I, G, S, L)$ for the following graph:



d^0	d^1
0.6	0.4

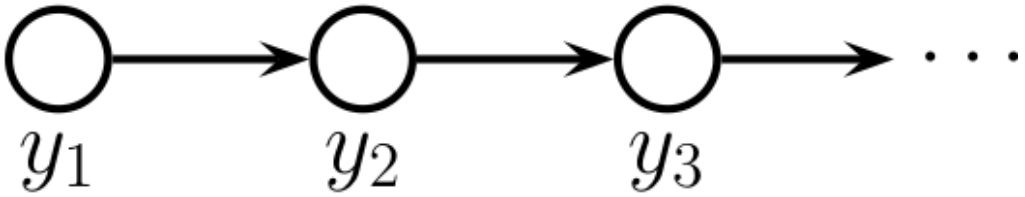
i^0	i^1
0.7	0.3

	g^1	g^2	g^3
i^0, d^0	0.3	0.4	0.3
i^0, d^1	0.05	0.25	0.7
i^1, d^0	0.9	0.08	0.02
i^1, d^1	0.5	0.3	0.2

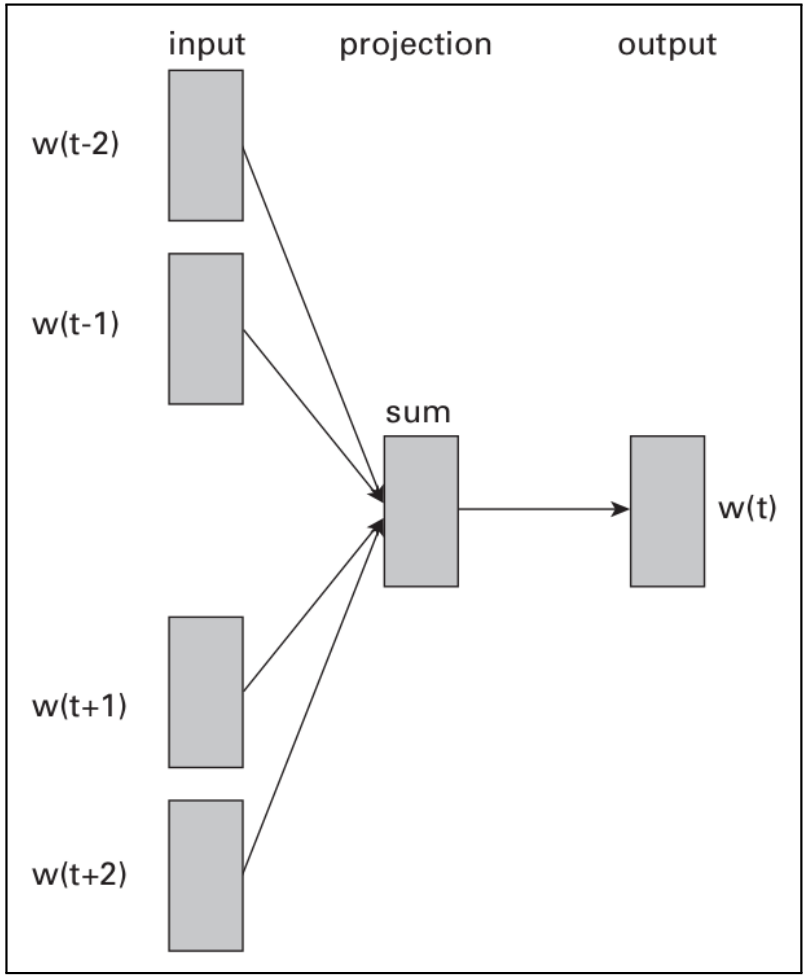
	s^0	s^1
i^0	0.95	0.05
i^1	0.2	0.8

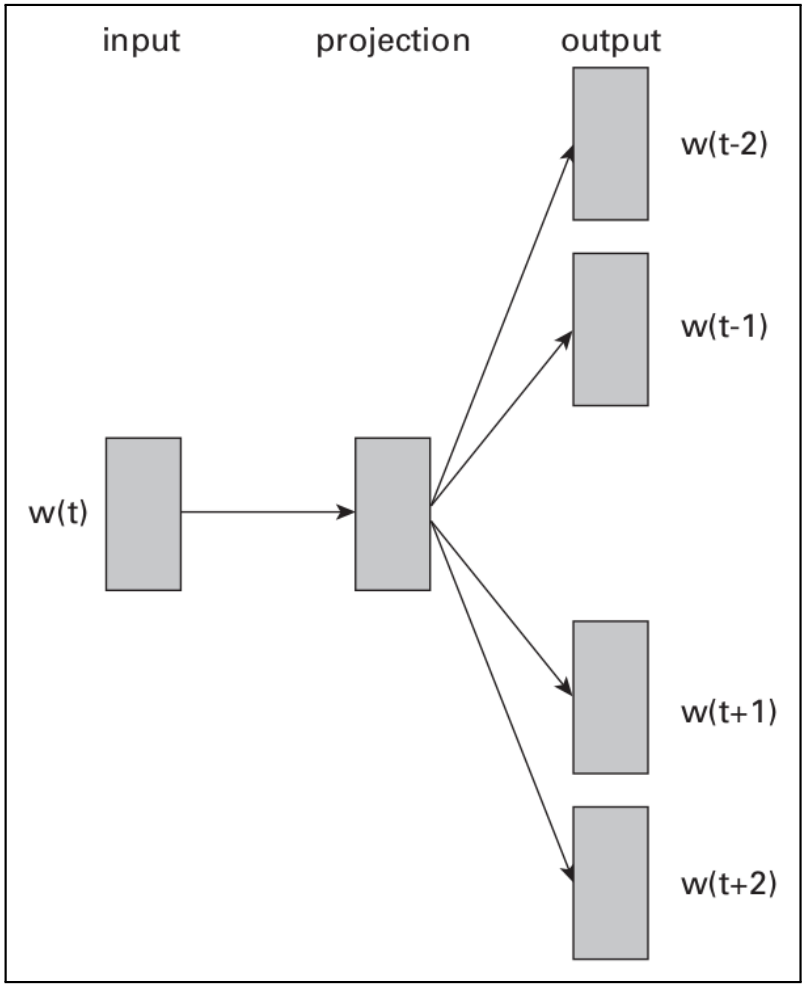
	l^0	l^1
g^1	0.1	0.9
g^2	0.4	0.6
g^3	0.99	0.01

Q. Compute the $p(y_1, y_2, \cdots, y_T)$ for the following graph:



Q. Compute the $p(w_{t-2}, w_{t-1}, w_t, w_{t+1}, w_{t+2})$ for the following graphs:





Word2vec [MCC+13]

shallow NNs (u or v) for predicting a word given its context

- CBOW (continuous BOW); $\bar{v}_t$ ($\sim$ BOW) as a context

$$\log p(w_{1:T}) = \sum_{t=1}^{T} \log p(w_t \mid w_{J_t}) \qquad (J_t = [t-m, t+m] \backslash \{t\})$$

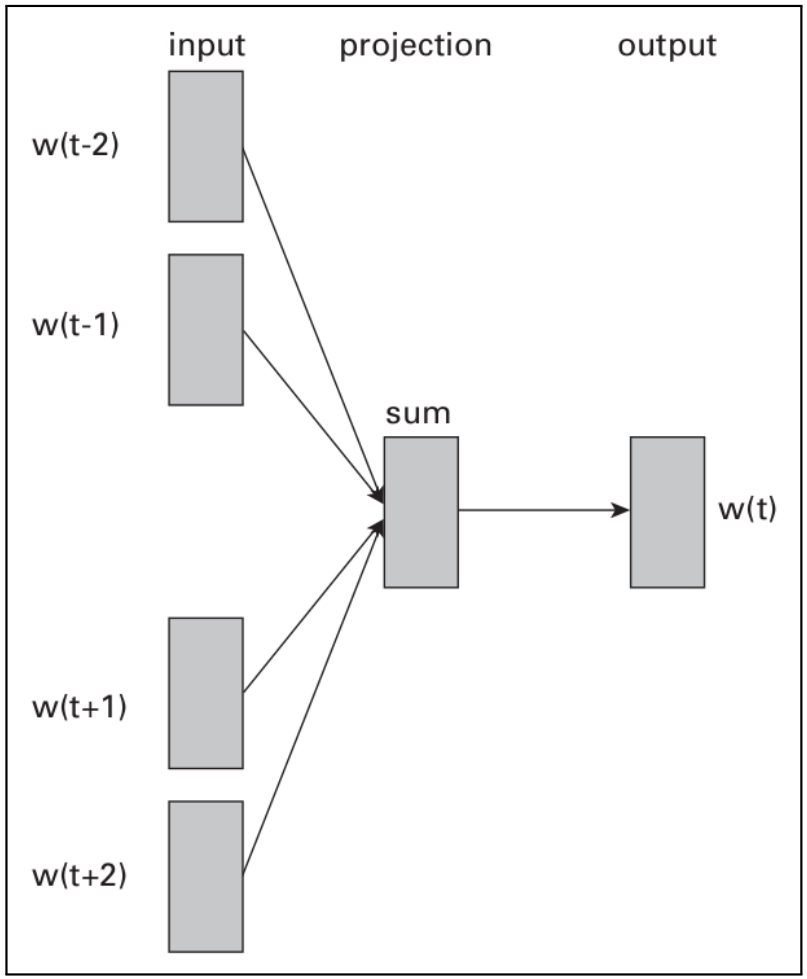
$$= \sum_{t=1}^{T} \log \frac{\exp(v(w_t)^\top \bar{v}_t)}{\sum_{w'} \exp(v(w')^\top \bar{v}_t)}$$

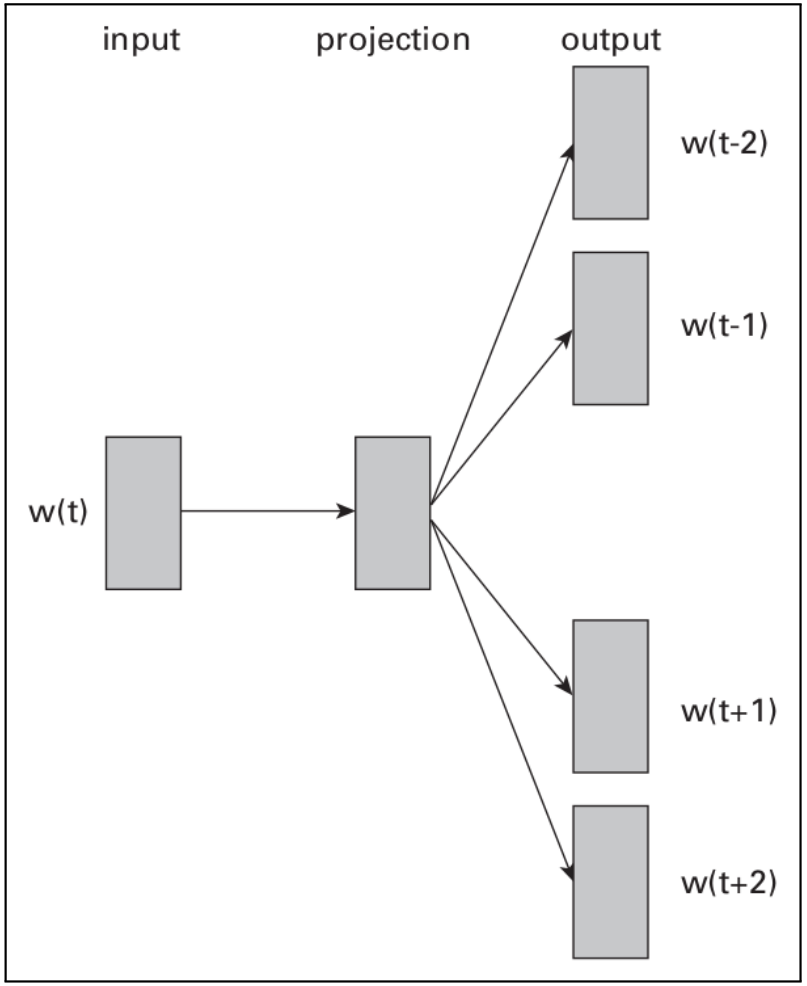
$$\bar{v}_t = \frac{1}{2m} \sum_{j \in J_t} v(w_j)$$

- skipgram (u: input embedding, v: target embedding)

$$\log p(w_{1:T}) = \sum_{t=1}^{T} \sum_{j \in J_t} \log p(w_j \mid w_t)$$

$$= \sum_{t=1}^{T} \sum_{j \in J_t} \log \frac{\exp(v(w_j)^\top u(w_t))}{\sum_{w'} \exp(v(w')^\top u(w_t))}$$





$$\log \frac{\exp(v(w_t)^\top \bar{v}_t)}{\sum_{w'} \exp(v(w')^\top \bar{v}_t)} \text{ and } \sum_{j \in J_t} \log \frac{\exp(v(w_j)^\top u(w_t))}{\sum_{w'} \exp(v(w')^\top u(w_t))}$$

$$\text{context } (\bar{v}_t \text{ and } u(w_t)) \mapsto \text{prediction } (v(w_t) \text{ and } \{v_j\}_{j \in J_t})$$

- e.g., $V = 10000, K = 16, N = 25k$ (IMDB)
- $E \in \mathbb{R}^{16 \times 10000}$
- instead of learning the whole parameters in a supervised way, we can perform unsupervised pre-training of word embedding models (E)
- then, fine-tune the other parameters for a downstream task